

Technical Documentation (System Architecture Report) AI-Driven 3D Island Expedition

ISDN 3150 – Spring 2026

May 2026

Project Overview

This project implements a replayable, AI-driven 3D supernatural island expedition game. The core design goal is to make each run feel alive and reactive through dynamic events, adaptive pacing, and narrative variety while keeping the player engaged with objective progression and rewards. The build uses a web-first stack with a real-time 3D frontend and a Node.js AI service layer. The system is structured around an AI Director that controls pacing, weather, and encounters, and supporting agents that provide NPC dialogue, event selection, and reward decisions.

System Architecture

Frontend (React + Three.js + R3F). The runtime client is a React application that renders the world using React Three Fiber (R3F) and Three.js. Core gameplay is implemented as components and Zustand stores:

- **Scene and rendering:** The 3D scene is rendered inside a R3F Canvas, with terrain, lighting, and weather layers.
- **Player and NPCs:** The player avatar and NPCs use VRM models loaded via GLTF + VRM loader. Movement, collisions, and animation logic live in the player rig component.
- **Game loop and UI:** Objective tracking, extraction logic, run outcomes, and minimap markers are handled in dedicated UI and game components.
- **State management:** World, run, NPC, and UI state are managed via Zustand stores to keep the render loop responsive.

Backend (Node.js + Express). The AI service layer is a Node.js Express API that exposes agent endpoints:

- `/api/ai/director` for pacing, weather, and event selection
- `/api/ai/npc` for dialogue, emotion, and suspicion

- `/api/ai/event` for event prefab triggers
- `/api/ai/reward` for card rewards and unlock rules

Each endpoint delegates to a dedicated agent module and validates outputs with JSON Schema via AJV before returning them to the client.

AI Pipeline and Multi-Agent Roles

The AI layer is implemented as a multi-agent system with specialized responsibilities and strict JSON outputs to keep the game runtime deterministic and safe. Each agent is independently prompted and validated, then orchestrated by the game loop to drive distinct parts of the experience (pacing, dialogue, events, and rewards).

- **Director Agent:** Generates next event, weather, pacing, and world adjustments. It reacts to world state (danger, corruption, recent events) to ensure tension escalation with occasional calm moments.
- **NPC Agent:** Generates dialogue plus character state (suspicion, emotion, tone) based on player history and faction trust.
- **Event Agent:** Selects a prefab id and associated effects for dynamic encounters.
- **Reward Agent:** Chooses collectible card rewards and unlock conditions based on run outcomes.

Prompt Engineering Strategies

Prompt engineering is intentionally structured, multi-layered, and schema-driven:

- **Role anchoring:** Each agent receives a role definition and responsibilities (for example, the Director is told to manage pacing, atmosphere, and objective progression).
- **Phase-aware guidance:** The Director prompt defines early, mid, and late-run behavior to shape escalation in a controlled way.
- **Anti-repetition guardrails:** Prompts explicitly instruct the model to avoid repeating recent events and to vary locations.
- **Output-only constraints:** Prompts require strict JSON with named keys to prevent prose responses and reduce parsing failures.

This prompt design is coupled with strict schema validation to reduce runtime errors and enforce deterministic output shapes.

Function Calling and Structured Outputs

Instead of free-form text, the AI layer uses schema-constrained JSON outputs. In practice, this functions as a function-calling style contract:

- Each request includes a JSON Schema describing the expected output shape.

- The OpenAI Responses API is used with `response_format: json_schema`, ensuring only valid JSON is returned.
- AJV validates the returned payload before it is accepted by the game runtime.

This strategy provides strong guarantees that AI outputs match the engine's expectations, which is critical for real-time systems. It also minimizes brittle parsing logic in the client.

RAG (Retrieval-Augmented Generation)

The design goal includes user material grounding so that questions and objectives can be aligned with course notes. The intended RAG pipeline is:

1. **Ingestion:** User PDFs and notes are parsed into text and chunked into topic-aligned segments.
2. **Embedding:** Chunks are embedded and stored in a vector index (planned for Supabase + pgvector).
3. **Retrieval:** At runtime, the game queries the index using current objective context and player history to retrieve the most relevant chunks.
4. **Grounded generation:** Retrieved context is injected into prompts so the Director or puzzle generation can stay aligned with the user curriculum.

The current build focuses on the AI Director, NPC, event, and reward agents. The RAG pipeline is designed and aligned with the system architecture, and can be connected to the agent input stage without changing the runtime contract (JSON schemas remain the same).

Image Models and Visual Assets

Two types of visual content are used:

- **Card art:** Image generation is produced with ComfyUI, curated into themed sets that unlock characters.
- **3D characters:** VRM avatars are used for NPCs and the player. Models are loaded with the VRM loader and animated in the runtime.

This split keeps image generation in the offline pipeline (card creation) while the runtime focuses on stable 3D rendering.

Alignment With Design Goals

The technical pipeline reinforces the core design goals in four ways:

- **Replayability:** The Director Agent and event system create run-to-run variance in encounters, weather, and pacing.

- **Tension control:** Phase-aware prompts and world-state feedback allow gradual escalation instead of random spikes.
- **Objective-driven flow:** JSON outputs provide structured goals that the game loop can schedule and visualize.
- **Reward motivation:** The Reward Agent supports collectible card unlocks, tying gameplay progress to tangible rewards.

Course Tools Used

The system demonstrates the following course tools in a cohesive pipeline:

- **Prompt engineering:** Structured, role-based prompts with phase rules and anti-repetition constraints.
- **Function calling paradigm:** Schema-constrained JSON outputs validated by AJV, ensuring safe, deterministic AI control.
- **RAG (planned integration):** A vector retrieval pipeline designed to ground AI output in user materials.
- **Image models:** Midjourney for card art and VRM models for 3D characters.

Conclusion

This project delivers an AI-driven 3D exploration game with a clear separation of concerns between the runtime engine and AI decision making. The structured output strategy keeps the system stable, while the Director-led design provides pacing, replayability, and tension control. The planned RAG extension ensures the architecture can support curriculum-grounded content without changing the core engine contracts.